

Production Crash Audit & Prevention Framework

Status: P0 — SITE IS BROKEN IN PRODUCTION

The Crash: What Happened

The admin dashboard at mappingwithmelanin.com crashes with **React error #310** ("Objects are not valid as a React child"). The founder's profile shows blank data (no name, no badges, no saved places).

Root Cause Analysis

The Problem: Stale Compiled Bundle

The `dist/index.mjs` file that Railway deploys is **out of sync** with the source code and the database schema. Here's the proof:

What	In Source (<code>.ts</code> files)	In Compiled Bundle (<code>dist/index.mjs</code>)	In Railway Postgres
<code>visit_tip</code> column	✗ Not in Drizzle schema	✗ 0 references	✓ Exists, populated
<code>content_note</code> column	✗ Not in Drizzle schema	✗ 0 references	✓ Exists, populated
<code>pin_type</code> column	✗ Not in Drizzle schema	✗ 0 references	✓ Exists, populated
<code>listing_status</code> column	Referenced in source	1 reference in dist	✓ Exists, populated
<code>isTester</code> flag	Referenced in source	✗ 0 references in dist	✓ Exists in DB
<code>seed-manus</code> endpoints	In source	✗ 0 references in dist	N/A

The compiled bundle (`dist/index.mjs`) is 455,977 lines but is **MISSING** all recent **changes**. Railway runs this stale bundle. The database has been modified (new columns, new data), but the running code doesn't know about those changes. When the frontend tries to render data that includes new fields, React crashes.

Why This Keeps Happening

Replit's deployment process has a fundamental flaw:

1. They modify source code (`.ts` files)
2. They run migrations directly against Railway Postgres (adding columns, seeding data)
3. They commit the source changes
4. They push to GitHub
5. Railway triggers a build — **BUT the build uses a cached layer** that contains the OLD `dist/index.mjs`
6. The running binary is stale — it doesn't include the new routes, new schema, or new columns
7. The frontend (which WAS rebuilt) tries to use data from the new schema → **CRASH**

This is the **THIRD TIME** this session that Railway's build cache has caused a production issue. It's not a one-off — it's a systemic failure in the deployment pipeline.

Immediate Fix Required

Option 1 (Fastest): Rebuild dist locally and commit it

Bash

```
cd artifacts/api-server
pnpm run build
git add dist/index.mjs
git commit -m "fix: rebuild dist to match source — stale bundle causing product
git push
```

This forces Railway to deploy the fresh bundle because the file changed.

Option 2: Fix Railway's build pipeline

The `Custom Build Command` in Railway Settings is:

Plain Text

```
pnpm --filter @workspace/api-server run build
```

This SHOULD rebuild from source on every deploy. If it's not doing so, the nixpacks cache is serving a stale `node_modules/.cache` or `dist/` from a previous build layer.

Fix: Add `rm -rf dist &&` before the build command:

Plain Text

```
rm -rf dist && pnpm --filter @workspace/api-server run build
```

This ensures the dist directory is always rebuilt from scratch.

Option 3: Nuclear — Delete and redeploy the Railway service

If nothing else works, delete the api-server service in Railway and recreate it pointing to the same GitHub repo. This eliminates all cached layers.

Why the Profile Shows Blank

The profile page fetches user data from `/api/auth/user`. If the API returns fields that the frontend doesn't expect (or vice versa), React components crash or render blank. The stale binary may be:

- Not returning `first_name` / `last_name` because the query was modified in source but the bundle is old
- Returning null for fields that the frontend expects as strings
- The admin dashboard crash (React #310) suggests a `.map()` call on a null value — meaning an API response that should be an array is returning null/undefined

The fix is the same: rebuild the dist bundle so the running code matches the source.

Prevention Framework: Stop This From Ever Happening Again

Rule 1: NEVER modify the database without deploying matching code

If you add a column to Postgres, the Drizzle schema AND the compiled bundle must be updated in the SAME deployment. Never seed data into columns that the running code doesn't know about.

Rule 2: ALWAYS rebuild dist before pushing

Every commit that changes source code in `artifacts/api-server/src/` MUST include a rebuilt `dist/index.mjs`. The two-commit rule exists but isn't being followed — the "rebuild from HEAD" commit is rebuilding from the same stale cache.

Rule 3: Add a build verification step

After every Railway deployment, verify the running bundle:

Bash

```
curl https://api.melaninmaps.com/api/version
```

The response should include a `built_from_sha` that matches the latest commit. If it doesn't, the deployment is stale and must be fixed before any other work continues.

Rule 4: Add a health check that catches schema mismatches

Create a `/api/health/schema` endpoint that:

- Queries the database for column names on key tables
- Compares against what the Drizzle schema expects
- Returns a mismatch list if they differ
- This catches the "database was modified but code wasn't rebuilt" scenario

Rule 5: NEVER run migrations directly against production without deploying code first

The pattern should be:

1. Update Drizzle schema in source
2. Rebuild dist
3. Commit and push (code deploys)
4. THEN run migrations/seeds against production
5. Verify with `/api/version`

NOT:

1. Run migrations against production ← THIS IS WHAT KEEPS HAPPENING
2. Update source code
3. Push (but cache serves old binary)
4. Production crashes because code doesn't match DB

Rule 6: Pre-deployment checklist (mandatory before every push)

- ☐ `dist/index.mjs` was rebuilt from current source

- ☐ All new DB columns are in the Drizzle schema
 - ☐ `/api/version` returns the correct SHA after deploy
 - ☐ Admin dashboard loads without errors
 - ☐ Profile page loads with data
 - ☐ No React errors in browser console
-

Founder's Directive

This is the THIRD time Railway's build cache has caused a production issue in a single session. This is unacceptable. The fix must be:

1. **Immediate:** Rebuild dist, push, verify the site works (profile loads, admin loads, no crashes)
2. **Permanent:** Implement Rules 1-6 above so this NEVER happens again
3. **Accountable:** After fixing, provide a screenshot of the admin dashboard loading correctly AND the founder's profile showing her name

Do not proceed with ANY other work (Task #101, Build 103, anything) until the production site is stable and the founder can log in and see her profile data.

Timeline

- Fix the crash: **Within 1 hour**
- Implement prevention rules: **Within 24 hours**
- Provide verification screenshots: **Immediately after fix**